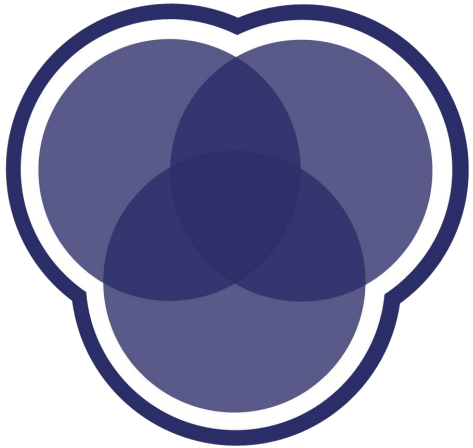
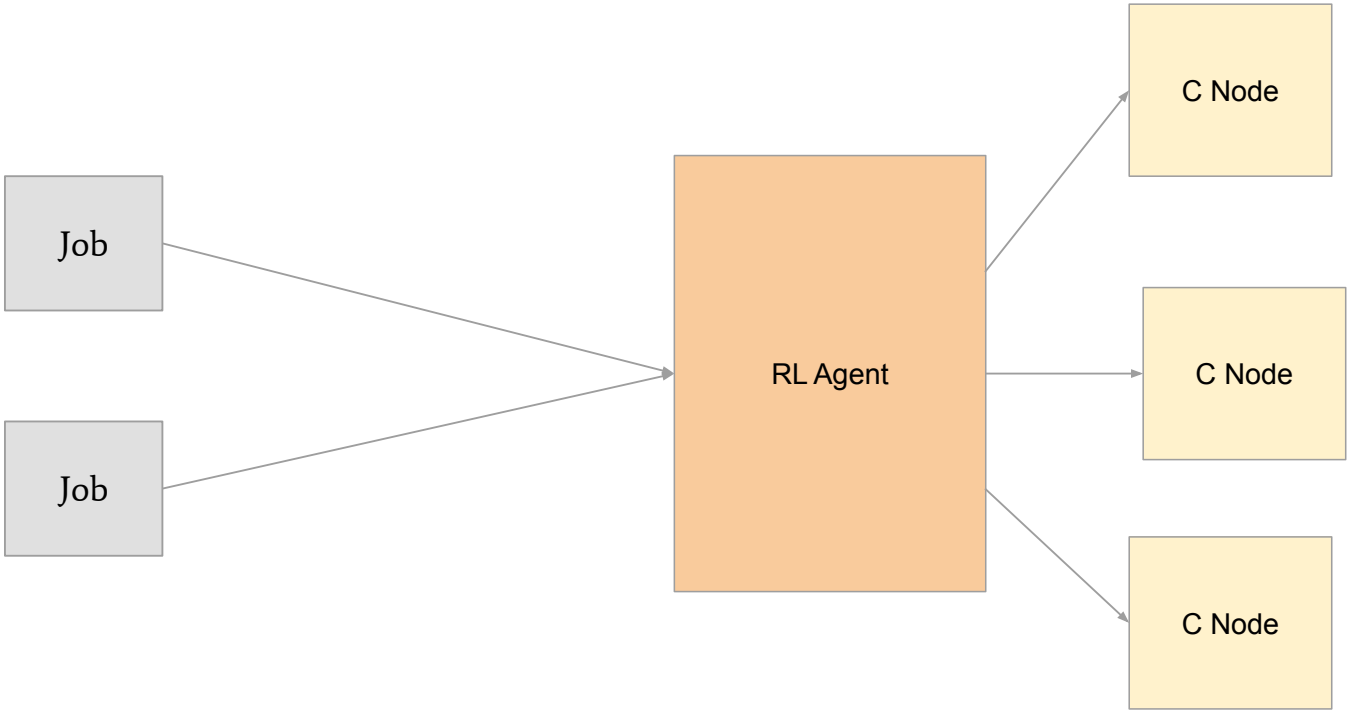


Slurm-Aware



Meeting 4: The Big Switch Up

Overview



Phases

Phase 1: Basics: Event Simulator (Discrete Time Event)

Phase 2: Basics: Minimal RL Agent(s), Evaluation

Phase 3: Having Fun: Adding Preemption and Reservation

Phase 4 (tentative): Getting Rusty: Incorporating with the Compute Cluster



Phase 1: Event Simulator (Discrete Time Event)

- Maintain a priority queue of events (sorted by time)
- Initialize environment by generating a workload, adding arrivals to event queue
- Pop all the events with the most recent time from the event queue
  - Add arriving jobs to the **job queue**
  - Perform some metric tracking for completions (i.e decrement # currently executing jobs)
- Output observation of current state of queue + cluster



Phase 1: Event Simulator (Discrete Time Event)

- Receive an action (schedule job X, NOP)
- If action schedules a job
  - Check the validity, update queue + cluster state, add the associated completion time to the event queue
- If action is a NOP
  - Go back to step one (therefore incrementing time)



Intro - Gymnasium



- Lets us define a standard environment interface for RL algorithm libraries

- We can define most of our RL formulation here (except the policy and RL algorithm)

```python
class GridWorldEnv(gym.Env):
    metadata = {"render_modes": ["human", "rgb_array"], "render_fps": 4}

    def __init__(self, render_mode=None, size=5):
        self.size = size  # The size of the square grid
        self.window_size = 512  # The size of the PyGame window

        # Observations are dictionaries with the agent's and the target's location.
        # Each location is encoded as an element of {0, ..., `size`}^2, i.e. MultiDiscrete([size, size]).
        self.observation_space = spaces.Dict(
            {
                "agent": spaces.Box(0, size - 1, shape=(2,), dtype=int),
                "target": spaces.Box(0, size - 1, shape=(2,), dtype=int),
            }
        )
        self._agent_location = np.array([-1, -1], dtype=int)
        self._target_location = np.array([-1, -1], dtype=int)

        # We have 4 actions, corresponding to "right", "up", "left", "down"
        self.action_space = spaces.Discrete(4)
```

Intro - Gymnasium




```python
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```



Phase 2: Basics: Minimal RL Agent(s)

Simple Design

- Agent can only add a job to run on cluster if there is currently space available
- Reward = - (num_jobs * change_in_time)

DeepRM Approach

- Main difference: placing a job for future execution is valid action
- Reward = slowdown, roughly

We'll jump to the DeepRM way



Phase 2: Basics: Minimal RL Agent(s)

What DeepRM Did

- Action Space
  - 10 job slots, NOP (selecting empty slot would be a NOP too)
- Observation Space
  - Cluster state up to X minutes into the future
  - Job information of jobs in 10 visible slots of queue
  - Backlog of jobs that are “further” back in queue
- Reward
  - Slowdown



Phase 2: Basics: Minimal RL Agent(s)

DeepRM Observation State

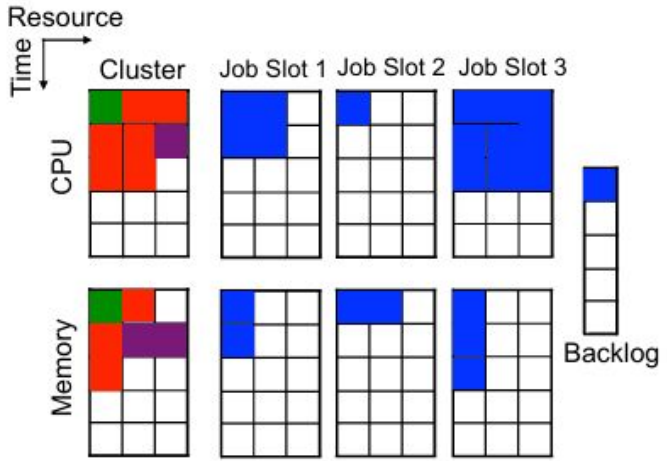


Figure 2: An example of a state representation, with two resources and three pending job slots.

Phase 2: Basics: Minimal RL Agent(s)

What DeepRM Did

- RL Algorithm
  - Custom REINFORCE implementation
- Policy
  - One hidden layer MLP



Intro - Stable-Baselines3

- Implements lots of different RL algorithms and policies, taking care of training for the most part
- Provides default MLP given observation and action spaces
- We can play around with MLP architecture and how observation is encoded as input

```python
import gymnasium as gym

from stable_baselines3 import PPO
from stable_baselines3.common.env_util import make_vec_env

# Parallel environments
vec_env = make_vec_env("CartPole-v1", n_envs=4)

model = PPO("MlpPolicy", vec_env, verbose=1)
model.learn(total_timesteps=25000)
model.save("ppo_cartpole")

del model # remove to demonstrate saving and loading

model = PPO.load("ppo_cartpole")

obs = vec_env.reset()
while True:
    action, _states = model.predict(obs)
    obs, rewards, dones, info = vec_env.step(action)
    vec_env.render("human")
```



Phase 2: Evaluation

I did not have time to get to this



Phase 3: Having Fun: Adding Preemption, Time Prediction

Coming next week :)



Phase 4: Getting Rusty with Compute Cluster

Coming next week :)

